Residency Is Not Reuse: ReduLink for Encrypted QUIC Object Streams

Michél Nguyen
Computer Science, University of the People
595 E. Colorado Boulevard, Suite 623, Pasadena, CA 91101, USA
ORCID: 0000-0001-6834-4422 | Corresponding author: minhmichel@outlook.de

Abstract

Encrypted transport prevents in-network redundancy elimination from seeing payloads, but endpoint reference substitution helps only when reusable state is both available and economically usable. We identify three necessary gates: authorized exact state remains resident, representation boundaries preserve referenceable bytes, and avoided literals amortize record, control, and repair cost. ReduLink instantiates this model as a binary FULL/REF codec on ordinary QUIC streams with a live TLS 1.3 exporter, actual stream identifiers, bounded true-LRU state, exact reconstruction, and batched miss repair. Across 40,872,024 IBM registry trace records, a 1 GiB per-client exact-blob cache yields only a 14.8% same-client warm-byte upper bound. On three pinned compressed container updates, ordinary whole-layer content addressing first avoids 26.65% of bytes; the remaining changed layers preserve at most 0.127% exact fixed-chunk bytes, so ReduLink expands that residual. Under deliberately aligned reuse, 16 shaped Reno conditions reduce client completion to 0.34-0.88 of raw QUIC, while first-byte latency is unchanged or worse. Same-connection multistream tests isolate small objects but slow the miss-heavy blocker, and synchronized flows expose a fairness cost. The result is a falsifiable deployment rule and a bounded mechanism, not a universal compression claim.

Keywords: QUIC; redundancy elimination; content addressing; shared state; transport evaluation; reproducible systems

1. Introduction

Redundant bytes remain redundant after encryption, but transparent middleboxes can no longer inspect them. Classic systems suppress repeated traffic on a link or across a network [1-3]. QUIC deliberately moves encryption, stream multiplexing, loss recovery, and congestion control into an endpoint-to-endpoint transport [14-18]. A compatible redundancy mechanism must therefore operate inside cooperating endpoints, before encryption and after decryption.

That placement is established prior art. EndRE operates before application encryption [4]; DOT transfers hash-named chunks and handles misses [5]; PACK and CoRE coordinate receiver or endpoint state [6,7]. The unresolved systems question is not whether a reference can replace bytes. It is when such a reference remains valid, aligned with the transmitted representation, and cheaper after metadata, repair, CPU, and transport behavior are counted.

This distinction matters in object delivery. Registries already use digest-addressed layers and transfer only absent blobs [33-35]. File trees favor rsync or content-defined chunking [8-10,22]. HTTP responses can use delta formats or Compression Dictionary Transport [11-13]. A new reference layer is justified only in the residual case where an authorized endpoint retains useful sub-object state, representation boundaries survive, and explicit repair is worth its cost.

We ask three research questions:

- RQ1: How often do residency and representation alignment hold in production-derived object workloads?
- RQ2: When aligned reuse is present, how do completion, first-byte latency, packet cost, multistream isolation, and competing-flow fairness compare with raw QUIC?
- RQ3: Which protocol and implementation costs determine the break-even boundary, and which security claims are actually supported?

The paper makes four contributions:

- A three-gate model that separates authorized residency, representation alignment, and byte economics. It explains why a high object revisit rate need not imply reusable transmitted bytes.
- A tested QUIC application-stream codec with live TLS exporter derivation, per-stream key separation, fixed binary records, bounded receive state, exact completion checks, and deterministic repair.
- An evidence ladder spanning a complete 40.87 million-record IBM registry trace, pinned compressed registry layers, 16 Linux kernel-shaped path conditions, same-connection multistream isolation, synchronized fairness, and CPU scaling through 32 MiB.
- Negative boundaries: ordinary layer CAS dominates the studied compressed updates; pre-encoding delays first byte; mixed encoded work reduces fairness; raw source trees favor rsync; and the HMAC is defensive state binding, not a second network security layer.

2. Related Work and Novelty Boundary

2.1 Redundancy elimination and delta transfer

Link and network-wide redundancy elimination established the potential for repeated-byte suppression [1-3]. EndRE is the closest placement precedent because it moves redundancy elimination to end systems [4]. DOT contributes content naming and explicit cache misses [5], while PACK and CoRE explore receiver-driven and cooperative endpoint state [6,7]. ReduLink does not claim novelty for endpoint placement, hash-named chunks, or miss repair.

Rsync, LBFS, and REBL show that boundary selection and delta structure matter [8-10]. FastCDC improves content-defined boundary selection [22]. HTTP delta encoding, VCDIFF, and RFC 9842 offer more appropriate abstractions for related HTTP responses [11-13]. Zstandard raw-content dictionaries are a strong baseline when the complete prior representation is retained [25,26,31]. Tentackle also carries dictionary and back-reference machinery over QUIC streams [30].

2.2 What is new

The novelty is the joint claim and its evidence: encrypted endpoint substitution is useful only after three independent gates are tested. Prior mechanisms often start after reusable state has been assumed. ReduLink makes that assumption measurable, maps failures to explicit fallback decisions, and evaluates QUIC-specific consequences with live exporter keying and actual stream IDs. The mechanism is intentionally narrow: a managed object protocol that already has authorized warm state but needs individually verified references and a bounded repair path.

3. Three-Gate Model

3.1 Necessary conditions

Let L be the bytes that the receiver must reconstruct. Let r references resolve immediately, f chunks be sent literally in the initial round, and m references miss and require literal repair. Let hR, hF, and hM denote the corresponding record or control costs, and let C0 collect fixed handshake-independent codec control. For chunk lengths li, the application-stream byte cost is:

$$W = C0 + r\,hR + f\,hF + sum(i\ in\ FULL)\ li + m(hM + hF) + sum(i\ in\ MISS)\ li$$

Strict byte benefit requires L / W > 1. This accounting gate is necessary but not sufficient. Gate 1 determines whether a candidate reference is authorized and resident, which controls m. Gate 2 determines whether the transmitted representation preserves the candidate boundary and bytes, which controls r before the protocol begins. A workload can pass any two gates and still fail the third.

Table 1. The three gates and their operational decisions.

Gate	Observable	Failure mode	Fallback
1. Authorized residency	Same-scope exact state under a byte-bounded policy	Miss, eviction, revocation, or wrong scope	FULL, raw transfer, or cache admission
2. Representation alignment	Exact reusable bytes after framing, compression, and object boundaries	Recompression or shifted boundaries erase matches	Whole-object CAS, delta codec, or different chunking
3. Economics	L / W including forward and reverse bytes	Metadata and repair exceed avoided literals	Raw or ordinary compression

The gates are deliberately representation-aware. A registry may revisit the same logical image while downloading different compressed layer bytes. Conversely, a warm exact blob can satisfy Gate 1 but add no value because whole-layer content addressing already suppresses its transfer. The model therefore tests existing coarse-grained reuse before adding fine-grained references.

3.2 ReduLink mapping

ReduLink is an application codec on a QUIC bidirectional stream, not a custom QUIC frame. HELLO declares version, chunk size, record count, output length, and SHA-256. Each ordered FULL record carries bytes; each REF names a warm chunk. END_ROUND closes the initial pass. The receiver returns one batched MISSING list, the sender validates every request and sends authenticated FULL repairs, and FINISH triggers exact length, sequence, and digest checks. A 13-byte symmetric FIRST_BYTE acknowledgement exists only in timing experiments and is excluded from protocol accounting.

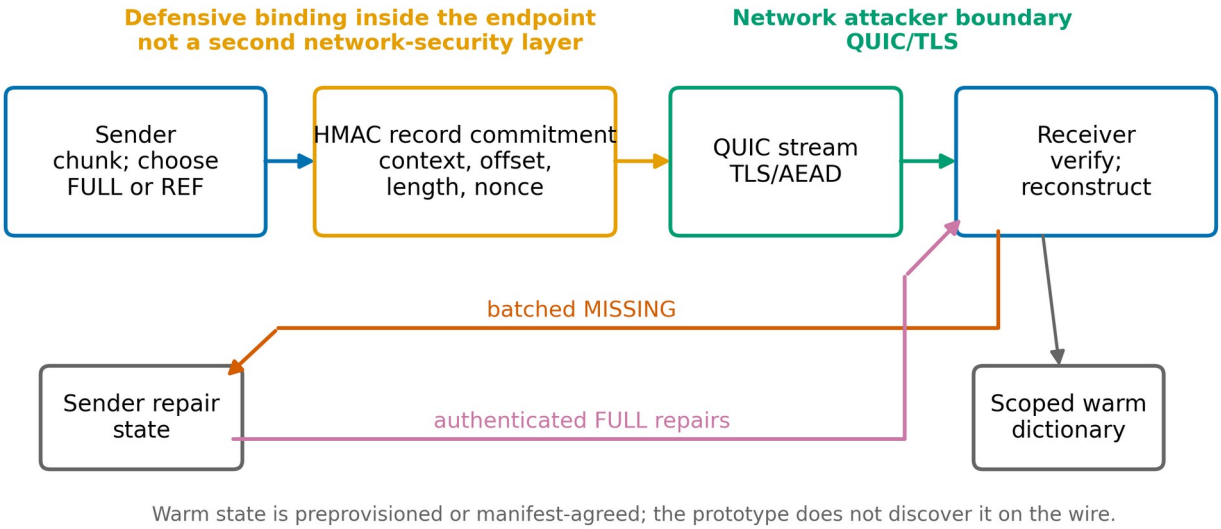

Figure 1. ReduLink mapping and trust boundaries. The repaired control and repair paths occupy separate lanes. QUIC/TLS is the network security boundary; HMAC commits endpoint representation state.

Table 2. Implemented messages and fail-closed checks.

Message	Direction	Purpose	Receiver or sender check
HELLO	C to S	Declare exact transfer	Version, quota, count, chunk size, length, digest
FULL / REF	C to S	Literal or warm-state step	Sequence, offset, length, nonce, scope, tag
END_ROUND	C to S	Close initial records	Exact declared count
MISSING	S to C	Batch unresolved REFs	Original REF, identifier, length, uniqueness
FULL repair	C to S	Supply missing literal	Pending sequence, offset, identifier, tag
FINISH	C to S	Finalize	Complete sequence, exact length, SHA-256

A record binds kind, epoch, UTF-8 scope, actual 62-bit QUIC stream ID, reconstructed offset, length, nonce, keyed 128-bit chunk identifier, and a 128-bit HMAC-SHA-256 tag. The native 16-byte scope yields 101 fixed bytes per FULL or REF before a FULL payload. Sender writes are drained in 64 KiB batches. REF hits and FULL inserts update true LRU. A heap-backed 4,096-entry nonce window rejects replay with bounded memory; a 100,000-nonce regression test checks the bound.

3.3 Keying and security boundary

QUIC/TLS provides confidentiality, integrity, peer transport authentication, loss recovery, and congestion control [14-16]. ReduLink independently invokes the TLS exporter at both endpoints with the private-use label EXPERIMENTAL-ReduLink-v1 and a canonical length-prefixed context [19,32]. HKDF derives direction- and stream-specific record keys [20,21]. Unequal exporter outputs abort. Every multistream experiment uses actual stream IDs 0, 4, 8, 12, and 16 in the derivation.

The HMAC is defensive state binding under standard pseudorandom-function assumptions [20,29]. It detects a valid representation step replayed into the wrong epoch, scope, direction, stream, or offset and detects stale dictionary bytes. It is not a second defense against a network attacker already handled by QUIC/TLS, and it does not protect a compromised endpoint. Deduplication and compressed length can reveal content existence; cross-user state is therefore excluded and deployments must partition state by authorization scope [13,23,24].

The prototype authenticates the server certificate but assumes application-level client authorization and an authenticated application session identifier. aioquic 1.3.0 exposes no public exporter API, so a version-gated bridge narrows the post-Server-Finished exporter master secret immediately to the ReduLink label [27]. This is live exporter evidence in one stack, not independent-stack interoperability or a mechanized composition proof.

4. Evaluation Methodology

4.1 Evidence ladder and workloads

Table 3. Evidence families and supported claims.

Study	Input and scale	Trials	Supported claim
IBM trace	2,791 JSON files; 40.87M records; 7 availability zones	Complete archive	Same-client exact-blob residency upper bound
Registry layers	Redis, httpd, Alpine pinned linux/amd64 updates	3 pairs x 3 chunk sizes	Whole-layer CAS and compressed-byte alignment
Kernel path	2 positive fixtures; 5/20 Mbit/s; 20/80 ms RTT; 0/0.5% loss	16 conditions x 20 pairs	Completion, TTFB, CPU, qdisc bytes and packets
QUIC streams	1 connection; 1 miss-heavy blocker + 4 warm-hit objects	20 paired rounds	Independent-stream completion isolation
Fairness	Raw/raw, ReduLink/ReduLink, calibrated mixed encoded work	20 rounds per case	Jain fairness of encoded goodput
CPU scaling	64 KiB through 32 MiB on localhost	10 pairs + 1 warmup	Python implementation cost and pre-encoding TTFB

The IBM archive is verified by SHA-1 06c4d412f85e2a307556cbf6c046002ebf6acc29. The source study collected 75 days from IBM Cloud registry deployments and released traces and a replayer [33]. We parse all 40,872,024 distributed records. Following the paper's classification, Dallas, London, Frankfurt, and Sydney are production; staging, prestaging, and development remain separate. A successful full-blob access is status-200 GET /blobs/ with positive response bytes. Per anonymized client and production center, an exact URI and observed size enter a true byte-bounded LRU. Because revocation and client deletion are absent, the result is an upper bound on authorized warm state, not a measured client cache hit rate.

The alignment study resolves three public tag pairs to immutable index and linux/amd64 manifest digests, downloads every referenced compressed blob through the Registry HTTP API, and verifies digest and length [34,35]. Identical layer digests are removed first as ordinary whole-layer CAS hits. Only changed compressed bytes are then divided into exact 1, 4, or 16 KiB chunks. Manifests, tag names, and retrieval times are recorded; tags are provenance only, never identity.

The shaped-path fixtures intentionally pass Gates 1 and 2: a deterministic constructed update and an author-constructed aligned fixture derived from hash-pinned public Redis release bytes. They test transport behavior under known reuse, not workload prevalence. The Linux workflow runs each method in alternating order on an isolated loopback namespace shaped by tc/netem [36]. Every transfer uses a fresh server-authenticated aioquic 1.3.0 connection and Reno. Client completion and TTFB use one monotonic client clock from immediately before connect to exact completion or receipt of FIRST_BYTE. Root-qdisc deltas include encrypted handshake, ACK, close, loss, and the 13-byte timing acknowledgement.

4.2 Pairing, statistics, and exactness

For each shaped condition, 20 paired raw/ReduLink ratios are computed before aggregation. The multistream experiment compares concurrent and sequential execution of the same ordered objects on one connection. The fairness barrier releases only after both connections are established and ReduLink preparation is complete, immediately before first writes. The mixed case calibrates 814 KiB raw against 8 MiB reconstructed ReduLink so encoded application work differs by only 0.009%. Jain's index is computed over encoded application-stream goodput [37].

Reported interval bars are deterministic 95% percentile-bootstrap intervals over paired ratios or per-round metrics, using 5,000 resamples and a fixed seed [38]. CPU scaling also reports median intervals because its 4 and 16 MiB rows are skewed. Deterministic byte profiles and fixed public artifacts are reported exactly rather than assigned sampling intervals. Every accepted row reconstructs exact bytes and SHA-256; source-tree rsync additionally reconstructs a canonical path/type/content manifest.

5. Production Gates: Residency Is Not Alignment

5.1 Same-client exact-blob residency

The four production registries contain 9,897,809 successful full-blob GETs and 142,417,129,344,189 served bytes. At 64 MiB per client, 8.7% of requests but only 1.6% of bytes find the same exact blob resident. At 256 MiB the bounds are 15.2% and 8.8%; at 1 GiB they reach 23.7% and 14.8%. Large blobs and churn make byte reuse substantially lower than request reuse. The trace supports Gate 1 only; it contains identifiers and sizes, not payload bytes.

5.2 Compressed representation alignment

The three pinned updates contain 105,936,340 update-layer bytes. Whole-layer CAS already avoids 28,230,334 bytes (26.65%), leaving 77,706,006 changed compressed bytes. Within that residual, exact fixed-chunk matches are only 0.127% at 1 KiB, 0.111% at 4 KiB, and 0.021% at 16 KiB. ReduLink's residual multipliers are 0.908, 0.976, and 0.994; all are below break-even because records cost more than the surviving matches.

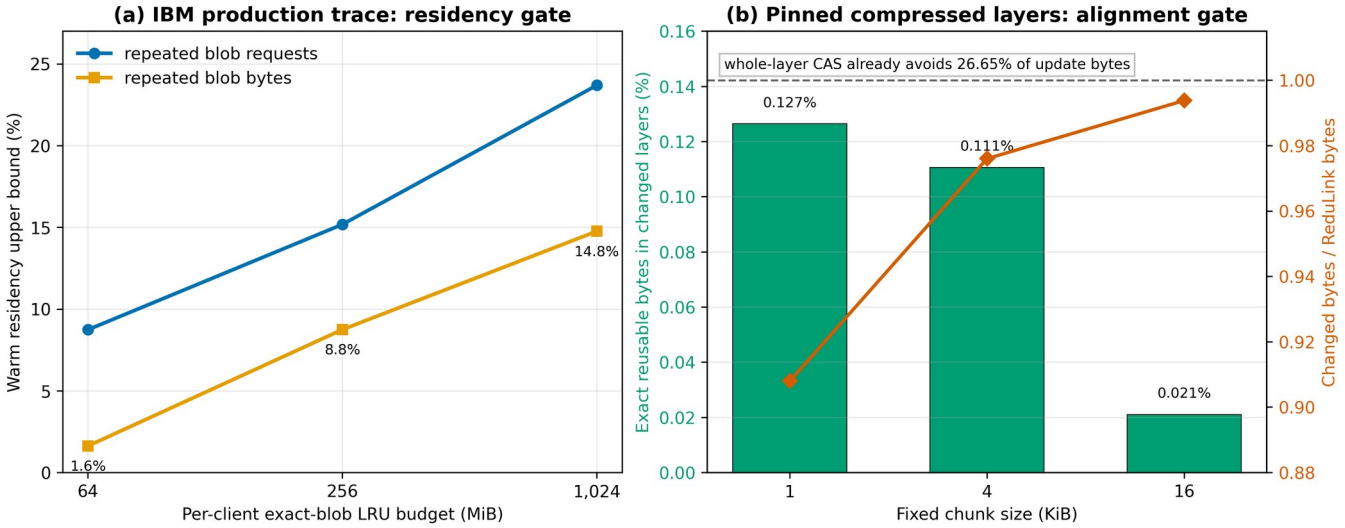

Figure 2. Gates 1 and 2. Left: exact same-client blob residency is an upper bound and bytes lag requests. Right: after whole-layer CAS, changed compressed layers retain almost no fixed-chunk identity; a multiplier below 1 means ReduLink expands the residual.

This negative result is the paper's central deployment lesson. Logical version similarity and repeated pulls do not imply repeated compressed bytes. A registry should retain digest-addressed layer CAS and should not add ReduLink below that boundary for these pairs. Fine-grained substitution remains plausible only for an aligned representation, such as stable uncompressed blocks or an application format designed around reusable objects.

6. Controlled Transport Behavior

6.1 Completion improves; first byte does not

For the constructed positive control, mean client completion is 0.58-0.88 of raw QUIC; seven of eight 95% intervals lie strictly below 1 and the noisiest 20 Mbit/s, 80 ms, 0.5% loss condition overlaps break-even. The qdisc-byte ratio remains 0.354-0.360. For the larger Redis-derived fixture, completion is 0.34-0.85 and all eight intervals are below 1; qdisc bytes are 0.274-0.277. These reductions follow encoded bytes through a real Linux queue and QUIC recovery, rather than assuming a stream-byte multiplier equals latency.

TTFB exposes the opposing cost. The positive control ranges from 0.98-1.12 of raw; five intervals are above 1 and three overlap 1. The Redis-derived fixture ranges from 1.08-1.37, with every interval above 1. The implementation computes chunk choices, tags, and warm dictionaries before application writes, while raw QUIC can expose offset zero immediately. Fewer bytes improve completion but do not compensate for pre-encoding at first byte.

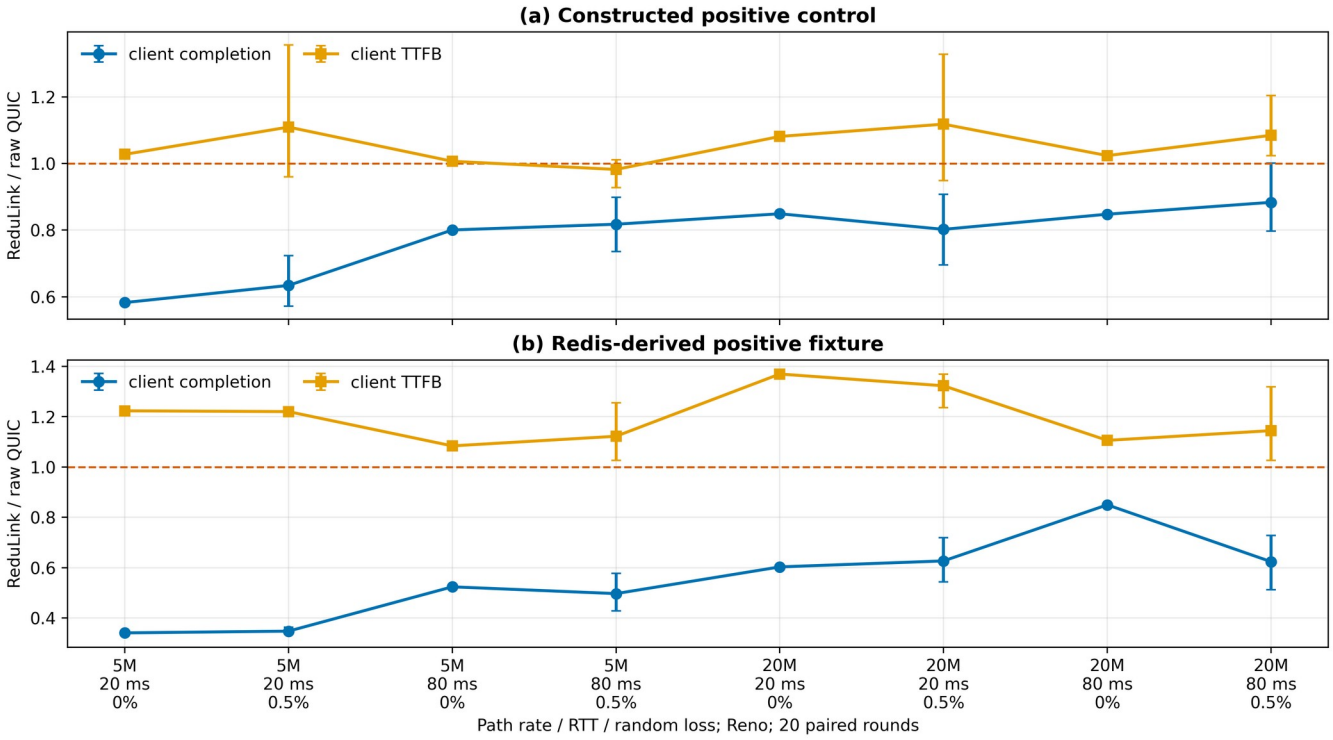

Figure 3. Client-clock completion and TTFB across 16 isolated Linux tc/netem conditions. Points are means of paired ReduLink/raw ratios; bars are 95% bootstrap intervals. Values below 1 favor ReduLink.

6.2 QUIC stream isolation and fairness

Multiplexing provides a genuinely QUIC-specific benefit. Four 64 KiB warm-hit objects and one 2 MiB miss-heavy blocker share one live connection. Concurrent streams reduce mean small-object completion to 0.076 of sequential execution (95% interval 0.069-0.083); all 80 small streams finish before their blocker. The trade-off is explicit: blocker completion rises to 1.164 (1.024-1.343), while session completion at 1.059 has an interval spanning 1. Independent streams isolate small-object completion, but they do not create free capacity.

Fairness also weakens. Mean Jain fairness is 0.967 for raw/raw, 0.931 for ReduLink/ReduLink, and 0.917 for calibrated raw/ReduLink. The mixed 95% interval is 0.890-0.943. All cases share the same 10 Mbit/s, 40 ms, 0.5% loss Reno bottleneck. The result is a cost to report, not evidence that reconstructed goodput should be used to claim fairness.

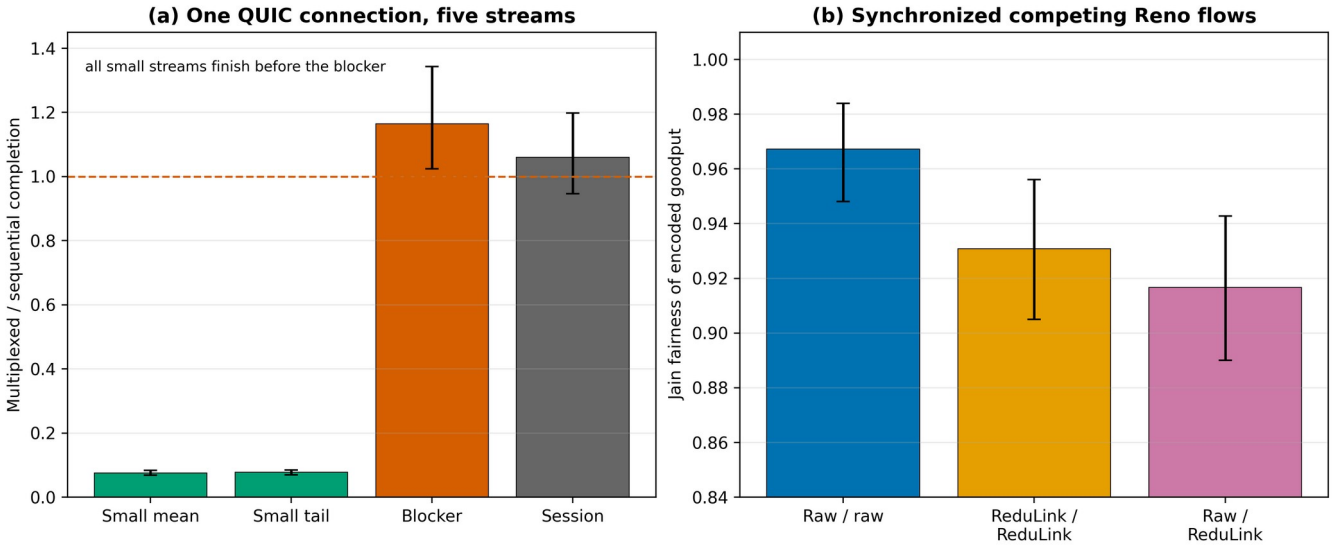

Figure 4. QUIC-specific behavior. Left: multiplexing isolates small streams but delays the blocker. Right: Jain fairness of encoded application goodput falls for ReduLink and the calibrated mixed case. Bars show 95% intervals.

6.3 CPU scaling and pre-encoding

The paired localhost study scales from 64 KiB to 32 MiB with a fresh verified connection per transfer. At 32 MiB, median completion and combined endpoint process CPU are 0.903 and 0.903 of raw QUIC; encoded stream bytes are 0.099. The result shows no hidden 16 or 32 MiB CPU cliff after replacing the old replay-window scan with a bounded heap-backed window. It does not predict an optimized native implementation.

First-byte behavior scales poorly because preparation is whole-object. Median raw versus ReduLink TTFB is 12.8 versus 265.7 ms at 16 MiB and 16.5 versus 513.3 ms at 32 MiB. Incremental chunk production is therefore required before latency-sensitive deployment.

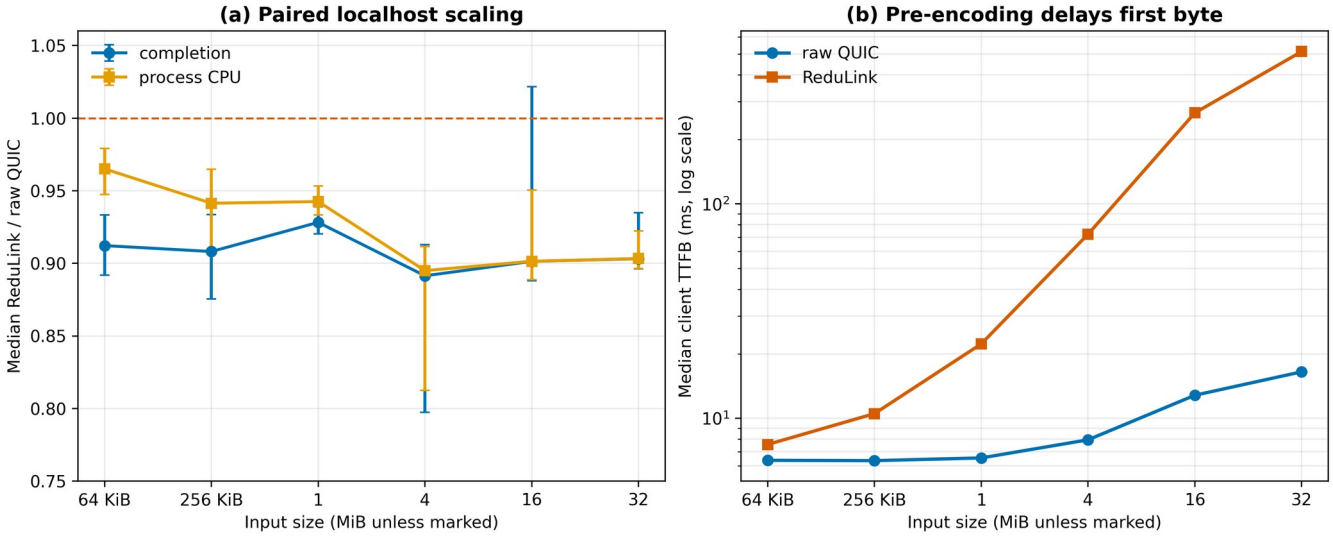

Figure 5. Local implementation scaling. Left: paired median completion and process CPU ratios with 95% median intervals. Right: absolute median client TTFB shows the cost of whole-object pre-encoding.

7. Byte Economics and Baselines

7.1 Exact break-even under semantic misses

The fixed 98,304-byte native profile makes Gate 3 executable. With 90 initial references and a 1 KiB chunk, every semantic miss adds 1125 forward bytes and 24 reverse bytes. The measured profile is exactly:

$$B(m) = 15\,914 + 1149\,m$$

Strict byte benefit holds through 71 of 90 misses and first fails at 72. This equation is profile-specific, but the method generalizes: deployments can substitute their scope length, record format, chunk distribution, and repair encoding before enabling references. Capacity is independent. At 16 MiB, an 8,192-chunk matched budget thrashes and yields 0.91x; a 24,576-chunk budget retains the working set and yields 10.10x.

7.2 Closest practical baselines

Table 4. Public release pairs. Multipliers are reconstructed bytes divided by each method's stated bytes; all decoders reconstruct exactly.

Pair	ReduLink named objects	Whole-object CAS	zstd prior stream	rsync raw tree
Click	1.81x	1.65x	58.11x	6.67x
Redis	7.48x	5.52x	4.62x	103.17x
Nginx	3.92x	3.16x	6.01x	56.85x

No method dominates. ReduLink reduces named-object bytes on Click, Redis, and Nginx, but the simpler 4 KiB token profile is smaller in every such row. Whole-object CAS is competitive when objects are unchanged, and zstd is much stronger on Click and Nginx. The rsync column uses raw file-tree payload and total bidirectional rsync bytes, not the named-object serialization, so it is a workload-choice result rather than a same-layer contest. On raw source-tree serialization, ReduLink is only 0.97-0.98x while rsync is 6.67-103.17x. An independently compressed control also expands. These results bound ReduLink to managed, aligned object streams needing explicit per-chunk repair.

8. Deployment and Security Discussion

8.1 Decision procedure

A deployment should proceed coarse to fine. First, apply native whole-object CAS. Second, estimate authorized, scope-partitioned residency under the actual byte budget. Third, measure exact matches on the transmitted representation, not on source trees or logical versions. Fourth, compute W including reverse repair. Finally, measure TTFB and shared-bottleneck behavior for the target transport. Failure at any stage selects a simpler mechanism rather than a tuned ReduLink threshold.

Table 5. Mechanism selection after the three gates.

Workload	Preferred mechanism	Why
Digest-addressed identical object	Whole-object CAS	Lowest reference overhead
Related file tree	rsync / file delta	Rolling alignment and metadata
Prior response or byte stream	CDT / zstd / VCDIFF	Codec-level differencing
Managed aligned QUIC object stream	ReduLink candidate	Explicit checked references and repair
No authorized warm state or high entropy	Raw / ordinary compression	References cannot amortize

8.2 Failure handling and privacy

The receiver rejects wrong scope, stream, epoch, direction, order, offset, nonce, length, tag, identifier, quota, final length, or digest before accepting completion. A REF absent from an otherwise valid dictionary becomes a semantic miss; a present but inconsistent entry is corruption and fails closed. The sender rejects duplicate, out-of-range, wrong-length, or non-REF repair requests. Public vectors independently reproduce canonical encoding, HKDF, identifiers, and tags.

Authorization remains external. A production system must define dictionary discovery, admission, revocation, synchronization, migration, and 0-RTT behavior. Content-existence and length oracles remain possible even with valid HMACs [13,23,24]. Per-origin or per-tenant partitioning, rate limits, minimum object sizes, and padding are policy requirements. ReduLink should not share global cross-user state.

9. Limitations and Threats to Validity

The production trace exposes anonymized request identity, URI, status, timestamp, and response bytes, not payload content, client cache deletion, or authorization changes. Its LRU values are upper bounds. The compressed-layer study contains only three recent public update pairs and fixed chunking. It directly falsifies a broad registry claim but cannot estimate all registries or representations. Content-defined chunking may alter Gate 2 and requires a separate evaluation [22].

All live transport experiments use one aioquic implementation and single-host Linux or macOS paths. tc/netem exercises a kernel queue and real QUIC loss recovery, but it is not a multi-host Internet deployment. Reno is held fixed; Cubic, BBR, mobile links, path migration, NAT rebinding, and independent stacks remain untested. Hosted runner hardware is not controlled. Twenty pairs support bounded condition comparisons, not population-wide latency claims.

The prototype buffers the object plan before first application write and buffers reconstruction until FINISH. This explains the TTFB cost and precludes an incremental-delivery claim. Reconstructed bytes do not consume QUIC flow-control credit; encoded bytes do. The fairness calibration equalizes encoded work, not reconstructed value. Python process CPU includes both in-process endpoints and is implementation evidence only.

The security analysis is conventional rather than mechanized. HMAC and HKDF use standard constructions [20,21,29], but tests do not prove composition, memory safety, or endpoint security. The live exporter bridge is private and version-specific. Client authorization and on-wire warm-state admission are assumed. These limitations keep the claim at a reproducible research prototype, not a deployable standard.

10. Reproducibility

The public artifact records scripts, dependency locks, immutable manifests, raw per-round CSV/JSON, summary intervals, queue counters, figures, and the manuscript builder [28]. The workflow evidence was generated from commit 81b56910cab554f28742b292c57dac9046ddb2c5. The full Linux run is GitHub Actions run 29495359131; all three jobs passed and each artifact records the same source commit. The IBM archive and every registry blob are independently digest checked. External payload caches are not redistributed.

Validation checks exact reconstruction, paired order, live exporter agreement, actual stream IDs, congestion-control selection, qdisc counters, source ancestry, regenerated figures, normalized DOCX content, PDF claims, citation coverage, and manuscript hashes. Deterministic evidence is compared exactly; environment-sensitive rsync control bytes have a documented tolerance while semantic counters and manifests remain exact. Historical timing artifacts are excluded from the submission set.

11. Conclusion

Residency is not reuse. Endpoint reference substitution over encrypted QUIC requires authorized resident state, byte-aligned representation, and positive economics. The IBM trace shows that request locality overstates reusable bytes; pinned compressed layers show that ordinary CAS can remove the useful coarse-grained reuse before fixed-chunk ReduLink begins. When all three gates are constructed to pass, ReduLink reduces shaped-path completion and QUIC streams isolate small objects, but first byte, blocker completion, and fairness expose real costs. The contribution is therefore a decision rule backed by positive and negative evidence, plus a bounded implementation that makes those decisions testable.

Data and Code Availability

Source code, manifests, generated evidence, figures, and validation scripts are available at https://github.com/pinkysworld/redulink-deduplex-quic [28]. SOURCE_COMMIT.txt binds the reported workflow evidence to its exact revision.

Declaration of Generative AI and AI-Assisted Technologies

During preparation, the author used OpenAI ChatGPT and Codex to support language editing, code review, benchmark validation, and document production. The author reviewed and verified the resulting text, code, data, references, and analyses and accepts full responsibility for the work.

References

[1] N. T. Spring and D. Wetherall, "A protocol-independent technique for eliminating redundant network traffic," Proc. ACM SIGCOMM, pp. 87-95, 2000. doi:10.1145/347059.347408.
[2] A. Anand, V. Sekar, and A. Akella, "SmartRE: An architecture for coordinated network-wide redundancy elimination," Proc. ACM SIGCOMM, pp. 87-98, 2009. doi:10.1145/1592568.1592580.
[3] A. Anand, A. Gupta, A. Akella, S. Seshan, and S. Shenker, "Redundancy in network traffic: Findings and implications," Proc. ACM SIGMETRICS, pp. 37-48, 2009. doi:10.1145/1555349.1555355.
[4] B. Aggarwal et al., "EndRE: An end-system redundancy elimination service for enterprises," Proc. 7th USENIX NSDI, 2010. https://www.usenix.org/conference/nsdi10-0/endre-end-system-redundancy-elimination-service-enterprises.
[5] N. Tolia, M. Kaminsky, D. G. Andersen, and S. Patil, "An architecture for Internet data transfer," Proc. 3rd USENIX NSDI, 2006. https://www.usenix.org/events/nsdi06/tech/full_papers/tolia/tolia_html.
[6] E. Zohar, I. Cidon, and O. Mokryn, "PACK: Prediction-based cloud bandwidth and cost reduction system," IEEE/ACM Trans. Netw., vol. 22, no. 1, pp. 39-51, 2014. doi:10.1109/TNET.2013.2240010.
[7] L. Yu, H. Shen, K. Sapra, L. Ye, and Z. Cai, "CoRE: Cooperative end-to-end traffic redundancy elimination for reducing cloud bandwidth cost," IEEE Trans. Parallel Distrib. Syst., vol. 28, no. 2, pp. 446-461, 2017. doi:10.1109/TPDS.2016.2578928.
[8] A. Tridgell and P. Mackerras, "The rsync algorithm," Australian National University, Tech. Rep. TR-CS-96-05, 1996. http://hdl.handle.net/1885/40765.
[9] A. Muthitacharoen, B. Chen, and D. Mazières, "A low-bandwidth network file system," Proc. ACM SOSP, pp. 174-187, 2001. doi:10.1145/502034.502052.
[10] P. Kulkarni, F. Douglis, J. LaVoie, and J. M. Tracey, "Redundancy elimination within large collections of files," Proc. USENIX ATC, pp. 59-72, 2004. https://www.usenix.org/conference/2004-usenix-annual-technical-conference/redundancy-elimination-within-large-collections.
[11] J. Mogul et al., "Delta encoding in HTTP," RFC 3229, IETF, 2002. doi:10.17487/RFC3229.
[12] D. Korn, J. MacDonald, J. Mogul, and K. Vo, "The VCDIFF generic differencing and compression data format," RFC 3284, IETF, 2002. doi:10.17487/RFC3284.
[13] P. Meenan and Y. Weiss, "Compression Dictionary Transport," RFC 9842, IETF, 2025. doi:10.17487/RFC9842.
[14] J. Iyengar and M. Thomson, "QUIC: A UDP-based multiplexed and secure transport," RFC 9000, IETF, 2021. doi:10.17487/RFC9000.
[15] M. Thomson and S. Turner, "Using TLS to secure QUIC," RFC 9001, IETF, 2021. doi:10.17487/RFC9001.
[16] J. Iyengar and I. Swett, "QUIC loss detection and congestion control," RFC 9002, IETF, 2021. doi:10.17487/RFC9002.
[17] M. Kuehlewind and B. Trammell, "Applicability of the QUIC transport protocol," RFC 9308, IETF, 2022. doi:10.17487/RFC9308.
[18] B. Trammell et al., "Manageability of the QUIC transport protocol," RFC 9312, IETF, 2022. doi:10.17487/RFC9312.
[19] E. Rescorla, "The Transport Layer Security (TLS) Protocol Version 1.3," RFC 8446, sec. 7.5, IETF, 2018. doi:10.17487/RFC8446.
[20] H. Krawczyk, M. Bellare, and R. Canetti, "HMAC: Keyed-hashing for message authentication," RFC 2104, IETF, 1997. doi:10.17487/RFC2104.
[21] H. Krawczyk and P. Eronen, "HMAC-based Extract-and-Expand Key Derivation Function (HKDF)," RFC 5869, IETF, 2010. doi:10.17487/RFC5869.
[22] W. Xia et al., "The design of fast content-defined chunking for data deduplication based storage systems," IEEE Trans. Parallel Distrib. Syst., vol. 31, no. 9, pp. 2017-2031, 2020. doi:10.1109/TPDS.2020.2984632.
[23] D. Harnik, B. Pinkas, and A. Shulman-Peleg, "Side channels in cloud services: Deduplication in cloud storage," IEEE Security & Privacy, vol. 8, no. 6, pp. 40-47, 2010. doi:10.1109/MSP.2010.187.
[24] M. Bellare, S. Keelveedhi, and T. Ristenpart, "DupLESS: Server-aided encryption for deduplicated storage," Proc. USENIX Security, pp. 179-194, 2013. https://www.usenix.org/conference/usenixsecurity13/technical-sessions/presentation/bellare.
[25] Y. Collet and M. Kucherawy, "Zstandard compression and the application/zstd media type," RFC 8878, IETF, 2021. doi:10.17487/RFC8878.
[26] Zstandard project, "Dictionary format and raw content dictionaries," documentation, version 1.5.7, accessed July 16, 2026. https://github.com/facebook/zstd/blob/v1.5.7/doc/zstd_compression_format.md.
[27] aioquic contributors, "aioquic: QUIC and HTTP/3 implementation in Python," version 1.3.0, software, accessed July 16, 2026. https://pypi.org/project/aioquic/1.3.0/.
[28] M. Nguyen, "ReduLink artifact and reproducibility package," source repository, 2026. https://github.com/pinkysworld/redulink-deduplex-quic.
[29] M. Bellare, R. Canetti, and H. Krawczyk, "Keying hash functions for message authentication," Advances in Cryptology, CRYPTO 1996, pp. 1-15. doi:10.1007/3-540-68697-5_1.
[30] Tentackle project, "TRIP over QUIC: the tentackle-quic module," implementation documentation, accessed July 16, 2026. https://tentackle.org/quic/.
[31] python-zstandard contributors, "zstandard Python bindings," version 0.25.0, software, accessed July 16, 2026. https://pypi.org/project/zstandard/0.25.0/.
[32] E. Rescorla, "Keying Material Exporters for Transport Layer Security (TLS)," RFC 5705, IETF, 2010. doi:10.17487/RFC5705.
[33] A. Anwar et al., "Improving Docker Registry Design based on Production Workload Analysis," Proc. 16th USENIX FAST, pp. 265-278, 2018. https://www.usenix.org/conference/fast18/presentation/anwar.
[34] Docker, "Registry HTTP API V2," documentation, accessed July 16, 2026. https://docs.docker.com/reference/api/registry/latest/.
[35] Open Container Initiative, "OCI Distribution Specification," documentation, accessed July 16, 2026. https://github.com/opencontainers/distribution-spec/blob/main/spec.md.
[36] Linux man-pages project, "tc-netem(8): Network emulator," manual page, accessed July 16, 2026. https://man7.org/linux/man-pages/man8/tc-netem.8.html.
[37] R. Jain, D. Chiu, and W. Hawe, "A quantitative measure of fairness and discrimination for resource allocation in shared computer systems," DEC Research Report TR-301, 1984.
[38] B. Efron and R. Tibshirani, An Introduction to the Bootstrap, Chapman and Hall/CRC, 1993. doi:10.1201/9780429246593.